

RedTrace

Simulador de moviment lateral en xarxes corporatives.
Estudi de Complexitat Algorítmica

Alumnes: Nicolau Mesalles, Gerard Grau, Hamza
ElHaddad, Oscar Saborido

ENTI-UB · CURS 2025-26

Taula de contingut

1. Introducció	2
2. Metodologia	2
2.1. Les tres notacions	2
2.2. Operacions bàsiques i el seu cost	2
2.3. Regles de composició i reducció	3
3. Variables del projecte	3
4. Anàlisi dels algorismes	3
4.1. DijkstraFinder — camí de mínim pes	3
4.2. BFSFinder — camí de mínims salts.....	4
4.3. CycleDetector — DFS amb 3 estats	4
4.4. AllPathsFinder — backtracking	5
4.5. Generador sintetic — generate_topology.....	5
4.6. IDSSimulator — deteccio per firmes.....	5
4.7. DecisionTreeRiskClassifier — arbre ID3	5
4.8. Informes i visualitzador.....	5
5. Resum de complexitats.....	6
6. Complexitat del pipeline complet	6
7. Validacio empirica	7
8. Conclusions.....	7
8.1. Que hem apres	7
8.2. Punts forts del codi.....	7
8.3. Coses que es podrien millorar.....	8
8.4. Veredicte.....	8

1. Introducció

RedTrace és un simulador de moviment lateral en xarxes corporatives que hem desenvolupat en Python 3.12 fent servir la llibreria NetworkX. La idea és simple: donada una topologia de xarxa (real o generada sintèticament), el programa calcula quina seria la millor ruta que podria seguir un atacant des d'un punt d'entrada fins a un objectiu concret.

A més de trobar camins, RedTrace classifica cada node de la xarxa per nivell de risc usant un arbre de decisió ID3, simula com respondria un sistema IDS i genera informes en format JSON i text.

En aquest document analitzem la complexitat algorítmica de tots els mòduls principals del motor. Seguim el mètode de quatre passos del Dr. David Berga (ADAA, ENTI-UB 2025-26): identificar blocs, analitzar cada bloc, compondre i reduir.

2. Metodologia

El mètode que fem servir té quatre passos clars:

Pas 1 — Identificar blocs: Separem el codi en unitats lògiques: assignacions, bucles, condicions i crides a funcions.

Pas 2 — Analitzar cada bloc: Assignem la complexitat de cada bloc per separat, en notació Big-O.

Pas 3 — Compondre: Combinem els blocs seguint tres regles: els blocs en seqüència es sumen, els anidats es multipliquen i els condicionals es queden amb el màxim.

Pas 4 — Reduir: Eliminem constants i ens quedem amb el terme dominant.

2.1. Les tres notacions

Notació	Significat	Com la fem servir
O (Big-O)	Cota superior (pitjor cas)	Temps màxim que pot trigar
Ω (Omega)	Cota inferior (millor cas)	Temps mínim garantit
Θ (Theta)	Cota ajustada (cas promig)	Quan O i Ω coincideixen

2.2. Operacions bàsiques i el seu cost

Tot el que no depèn de la mida de l'entrada és $O(1)$. Les operacions més habituals al codi:

Operació	Cost	Exemple al codi
Assignació	$O(1)$	<code>distances[entry] = 0.0</code>
Aritmètica / comparació	$O(1)$	<code>new_dist = dist + weight</code>
Accés a diccionari	$O(1)$	<code>graph._nodes[id]</code>
Inserció/cerca a set	$O(1)$	<code>visited.add(u)</code>
heappush / heappop	$O(\log n)$	Heap de fins a E entrades
<code>deque.append/popleft</code>	$O(1)$	<code>queue.popleft()</code> al BFS
Bucle de mida fixa k	$O(1)$	<code>for sig in signatures (k=4)</code>
Bucle de mida n	$O(n)$	<code>for nid in graph.node_ids</code>

2.3. Regles de composició i reducció

Regla	Quan s'aplica	Resultat
Seqüencial	Blocs un rere l'altre	$O(f_1 + f_2) \rightarrow \text{màxim}$
Anidada	Bucle dins de bucle	$O(f \cdot g)$
Condicional	if / elif / else	$O(\max(f, g))$
Eliminació de constants	Factor o sumand constant	$O(k \cdot f) = O(f)$
Dominància	Termes de creixements diferents	$O(f + g) = O(\max(f, g))$

3. Variables del projecte

Per simplificar la notació, definim les variables que fem servir al llarg de tot l'estudi:

Símbol	Significat	Notes
V	Nodes del graf	graph.num_nodes()
E	Arestes dirigides	graph.num_edges()
L	Longitud del camí trobat	$L \leq V$
k	Camins simples enumerats	Limitat per max_paths
S	Firmes IDS	$S = 4$ (constant)
n, f	Files i features del dataset ID3	$n \approx 200$, $f = 7$ (constants)
c	Cicles detectats	Depèn de la topologia

Suposem que l'accés a Node/Edge per id és $O(1)$ gràcies als diccionaris de Python, i que la suma dels graus de sortida de tots els nodes és exactament E .

4. Anàlisi dels algorismes

4.1. DijkstraFinder — camí de mínim pes

Dijkstra busca el camí de menor cost entre el node d'entrada i l'objectiu. Fa servir un heap mínim (heapq) per escollir sempre el node més proper pendent de visitar. El truc que fem servir és la lazy deletion: en lloc d'actualitzar el heap quan millorem una distància, simplement inserim l'entrada nova i descartem les antigues quan les extraiem. Això simplifica el codi i no canvia la complexitat.

```
def find_path(self, graph, entry, target):
    # Inicialització: tots els nodes a infinit        -> Theta(V)
    distances = {nid: inf for nid in graph.node_ids}
    distances[entry] = 0.0
    heap = [(0.0, entry)]

    while heap:
        dist, u = heapq.heappop(heap)                # fins V extraccions
        if u in visited: continue                    # O(log V)
        if u == target: break                         # lazy deletion
        for v in graph.neighbors(u):                 # Sum deg+(u) = E
            if new_dist < distances[v]:
                heapq.heappush(heap, (new_dist, v))  # O(log V)
    return self._reconstruct(...)                     # O(V)
```

Bloc	Cost	Per que
Inicialitzacio	$\Theta(V)$	3 comprensions de mida V
While + heappop	$O(V \log V)$	V extraccions · $O(\log V)$
For veins + heappush	$O(E \log V)$	E insercions totals · $O(\log V)$
Reconstruccio cami	$O(V)$	Recorre el cami $L \leq V$

Sumant i aplicant dominancia: $T(V,E) \in O((V+E) \log V)$

Cas	Complexitat	Condicio
Millor (Omega)	$\Omega(E + V \log V)$	El target es el primer que extraiem
Promig (Theta)	$\Theta((V+E) \log V)$	Topologia esparsa tipica
Pitjor (O)	$O(V^2 \log V)$	Graf dens, $E \approx V^2$
Espai	$O(V + E)$	distances + previous + heap

4.2. BFSFinder — cami de minims salts

El BFS explora la xarxa per nivells, garantint que el primer cami que arriba al desti es el que te menys salts. Cada node i cada aresta es visiten com a maxims una vegada, cosa que fa que sigui molt eficient per xarxes grans.

La clau és que el bucle intern no s'executa V vegades per cada iteracio exterior: en total, sumant tots els nodes, recorre exactament E arestes. Per tant no és $O(V \cdot E)$ sino $O(V+E)$.

$T(V,E) \in \Theta(V+E)$

Cas	Complexitat	Condicio
Millor (Omega)	$\Omega(1)$	entry == target directament
Promig (Theta)	$\Theta(V+E)$	Topologia connexa normal
Pitjor (O)	$O(V+E)$	Cal visitar tots els nodes
Espai	$O(V)$	Cua i diccionari de predecessors

4.3. CycleDetector — DFS amb 3 estats

El detector de cicles fa un DFS recursiu mantenint tres estats per cada node: no visitat, en curs (a la pila) i finalitzat. Si durant el recorregut trobem un veí que ja es 'en curs', és perquè hem tancat un cicle.

El cost base és el del DFS pur: $\Theta(V+E)$. El problema és que quan detectem un cicle, hem de buscar la seva posicio a la pila (stack.index), cosa que costa $O(V)$, i comprovar si ja el tenim guardat, que costa $O(c \cdot V)$ on c és el nombre de cicles ja detectats.

Sumant tot: $T(V,E,c) \in \Theta(V+E) + O(c^2 \cdot V)$

Cas especial: si el graf és aciclic ($c=0$), queda simplement $\Theta(V+E)$. Una millora senzilla seria guardar les normalitzacions en un set per baixar el cost extra a $O(c \cdot V)$.

4.4. AllPathsFinder — backtracking

Aquesta funció enumera tots els camins simples entre dos nodes. Fa servir backtracking: expandeix recursivament tots els veïns possibles i, quan arriba al destí, guarda el camí.

En el pitjor cas (graf complet sense cap limitació), el nombre de camins creix com una factorial: $O(V!)$. Per això tenim dos paràmetres de poda: `max_paths` (limita quants camins volem) i `max_depth` (limita la profunditat de la recursió). Amb `max_paths=K`, el cost baixa a $O(K \cdot V)$.

Configuració	Complexitat	Us recomanat
Sense poda	$O(V!)$	Xarxes molt petites ($V < 10$)
Amb <code>max_paths=K</code>	$O(K \cdot V)$	Producció — $K=50$ es suficient
Espai	$O(V)$	Pila de recursió + camí actual

4.5. Generador sintètic — `generate_topology`

Quan no tenim una topologia real, el generador crea una xarxa sintètica de n nodes. Primer assegura que el graf sigui connex creant un camí hamiltonià base, i després afegeix arestes aleatòries segons la densitat indicada.

El bucle que afegeix arestes per densitat és un doble bucle de $n \times n$, que domina tot el procés: $T(n) \in \Theta(n^2)$

Per $V=500$, això suposa 250.000 iteracions. Es manejable però és el motiu pel qual no convé generar topologies molt grans.

4.6. IDSSimulator — detecció per firmes

L'IDS avalua un camí contra un conjunt de 4 firmes predefinides. Cada firma recorre els L nodes del camí per comprovar una condició diferent (longitud, protocol, risc...). Com que el nombre de firmes és constant (4), per la regla d'eliminació de constants el factor 4 desapareix i ens queda: $T(L) \in \Theta(L) = O(V)$

4.7. DecisionTreeRiskClassifier — arbre ID3

El classificador té dues fases: entrenament (es fa una sola vegada) i inferència (es fa per cada node de la xarxa). Com que $n \approx 200$ i $f=7$ són constants del projecte, l'entrenament té cost constant i l'única variable real és V .

$T(V) \in \Theta(V)$ — lineal en el nombre de nodes a classificar.

4.8. Informes i visualitzador

Els generadors d'informe (JSON i text) recorren el camí i els cicles una vegada, cosa que dona $\Theta(L+c)$ per a tots dos.

El visualitzador és el **bottleneck** del sistema. L'algorisme `spring_layout` de `matplotlib` és iteratiu i té cost $O(I \cdot V^2)$ on $I=80$ iteracions. Domina tot el pipeline per a $V \geq 30$: $T(V) \in O(V^2)$

5. Resum de complexitats

Modul	Millor (Omega)	Promig (Theta)	Pitjor (O)	Espai
DijkstraFinder	$\Omega(E+V \log V)$	$\Theta((V+E) \log V)$	$O(V^2 \log V)$	$O(V+E)$
BFSFinder	$\Omega(1)$	$\Theta(V+E)$	$O(V+E)$	$O(V)$
CycleDetector	$\Omega(V)$	$\Theta(V+E)$	$O(V+E+c^2 \cdot V)$	$O(V+c \cdot V)$
AllPathsFinder	$\Omega(V+E)$	$O(K \cdot V)$ poda	$O(V!)$ sense poda	$O(V)$
generate_topology	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$	$O(n^2)$
IDSSimulator	$\Omega(1)$	$\Theta(L)=O(V)$	$O(V)$	$O(1)$
DecisionTree	$\Omega(\log n)$	$\Theta(V \cdot \log n)$	$O(V \cdot n)$	$O(n)$
JSONReport	$\Omega(L+c)$	$\Theta(L+c)$	$O(V+c)$	$O(L+c)$
TopologyViz	$\Omega(V^2)$	$\Theta(V^2)$	$O(V^2)$	$O(V+E)$

6. Complexitat del pipeline complet

Quan executem RedTrace de manera normal, les etapes s'encadenen seqüencialment:

Etapa	Operacio	Cost
1	Parsing + normalitzacio	$\Theta(V+E)$
2	TopologyGraph.build	$\Theta(V+E)$
3	Classificacio ID3 (V nodes)	$\Theta(V)$
4	DijkstraFinder (ShortestRoute)	$\Theta((V+E) \log V)$
5	CycleDetector	$\Theta(V+E) + O(c^2 \cdot V)$
6	IDSSimulator	$\Theta(L)$
7	JSONReport + TextReport	$\Theta(L+c)$
8	TopologyVisualizer	$O(V^2)$

Aplicant dominancia sobre tots els termes:

→ $\Theta(V+E)$, $\Theta(V)$, $\Theta(L)$ i $\Theta(L+c)$ queden absorbits per $\Theta((V+E) \log V)$.

→ $O(c^2 \cdot V)$ és significatiu només si hi ha molts cicles.

→ La visualitzacio $O(V^2)$ domina el pipeline si es genera el grafic.

Escenari	Complexitat total
Sense generar visualitzacio	$\Theta((V+E) \log V)$
Amb visualitzacio (matplotlib)	$O(V^2)$

7. Validacio empirica

Per confirmar que els resultats teorics son correctes, hem executat la suite de benchmarks (benchmarks/run.py) seguint les recomanacions del material de l'assignatura: usem `time.perf_counter()` per maxima precisió i repetim cada cas 3 vegades per absorbir el soroll del sistema operatiu.

```
def _time(fn, repeats=3):
    samples = []
    for _ in range(repeats):
        t0 = time.perf_counter()
        fn()
        samples.append(time.perf_counter() - t0)
    return statistics.mean(samples), statistics.stdev(samples)
```

La validacio visual es fa amb grafics log-log: si $T(V)$ pertany a $\Theta(V^k)$, la pendent de la recta hauria de ser aproximadament k .

Algorisme	Llei esperada	Pendent log-log
Dijkstra	$\Theta((V+E) \log V)$	$\approx 1 + \epsilon$
BFS	$\Theta(V+E)$	≈ 1
CycleDetector	$\Theta(V+E)$	≈ 1 (esparsa)
AllPaths (poda)	$O(K \cdot V)$	≈ 1
generate_topology	$\Theta(V^2)$	≈ 2

8. Conclusions

8.1. Que hem apres

La lliço mes important d'aquest estudi és que la composicio anidada en algorismes de graf no sempre dona $V \cdot V$. Quan en un bucle de V iteracions recorrem els veïns de cada node, el cost total no és $V \cdot \text{deg_max}$ sino simplement E , perque la suma de tots els graus és exactament el nombre d'arestes.

Gracies a l'eliminacio de constants, les 4 firmes de l'IDS i les 7 features de l>ID3 no apareixen a la complexitat final. I per la regla de dominancia, el bottleneck del pipeline no és cap dels algorismes de cerca sino el visualitzador matplotlib, que és $O(V^2)$.

8.2. Punts forts del codi

La **lazy deletion** a Dijkstra simplifica la implementacio i ens estalvia el decrease-key sense canviar la complexitat asimptotica.

L'encapsulació a **TopologyGraph** garanteix accessos $O(1)$ a nodes i arestes gracies als diccionaris interns.

La **poda** amb `max_paths` i `max_depth` a `AllPathsFinder` és el que fa viable l'us en produccio, evitant l'explosio factorial.

8.3. Coses que es podrien millorar

El `_is_duplicate` de `CycleDetector` té cost $O(c^2 \cdot V)$. Guardant les normalitzacions en un set podríem baixar-lo a $O(c \cdot V)$ sense gaire esforç.

El mètode `neighbors()` de `TopologyGraph` crea una nova llista a cada crida. Si retornes un iterador ens estalviariem la copia i el cost extra $\Theta(\deg(u))$.

Per a xarxes molt grans ($V \geq 500$), el **TopologyVisualizer** és el coll d'ampolla. Un layout $O(V \log V)$ o un renderitzat diferit millorarien molt el rendiment.

8.4. Veredicte

Les complexitats que vam anunciar al README del projecte — Dijkstra $O((V+E) \log V)$, BFS $\Theta(V+E)$, DFS $\Theta(V+E)$, AllPaths $O(V!)$, generador $\Theta(V^2)$ — coincideixen amb el que dona l'anàlisi formal pas a pas. No hem trobat cap divergència, i els benchmarks confirmen les pendents log-log esperades.